

Chapter 13 Complete Solutions - Tensors and Tensor Arithmetic

This solution set provides worked Python solutions for every problem in the Chapter 13 tensor homework assignment.

The main concepts are tensor shape, indexing, element-wise arithmetic, manual nested-loop implementation, tensor products, and a practical grayscale video tensor example.

The code is written in a teaching style with comments and printed explanations so Lexie can compare the mathematical idea to the NumPy implementation.

Complete Python Solution Code

```
# Chapter 13 Complete Solutions - Tensors and Tensor Arithmetic
# Student: Lexie
# Purpose: Worked solutions for tensor creation, indexing, arithmetic, tensor products,
#          and a practical grayscale video tensor problem.

import numpy as np

def section(title):
    print("\n" + "=" * 80)
    print(title)
    print("=" * 80)

def problem1_create_tensor():
    section("Problem 1 - Creating Tensors")

    T = np.array([
        [[1, 2, 3], [4, 5, 6]],
        [[7, 8, 9], [10, 11, 12]]
    ])

    print("T =")
    print(T)
    print("T.shape =", T.shape)

    levels, rows, columns = T.shape
    print("Number of levels =", levels)
    print("Number of rows per level =", rows)
    print("Number of columns per row =", columns)

    print("\nExplanation:")
    print("T is a 3D tensor. The first index selects the level, the second selects")
    print("the row inside that level, and the third selects the column inside that row.")

def problem2_tensor_indexing():
    section("Problem 2 - Tensor Indexing")

    T = np.array([
        [[1, 2, 3], [4, 5, 6]],
        [[7, 8, 9], [10, 11, 12]]
    ])

    print("First level T[0] =")
    print(T[0])

    print("\nSecond level T[1] =")
    print(T[1])

    value = T[1, 0, 2]
    print("\nT[1, 0, 2] =", value)

    print("\nExplanation:")
    print("T[1, 0, 2] means: level 1, row 0, column 2.")
    print("Python uses zero-based indexing, so level 1 is the second level,")
    print("row 0 is the first row, and column 2 is the third column.")

def problem3_tensor_arithmetic():
    section("Problem 3 - Tensor Arithmetic")
```

```

A = np.array([
    [[1, 2], [3, 4]],
    [[5, 6], [7, 8]]
], dtype=float)

B = np.array([
    [[10, 20], [30, 40]],
    [[50, 60], [70, 80]]
], dtype=float)

print("A.shape =", A.shape)
print("B.shape =", B.shape)

print("\nA + B =")
print(A + B)

print("\nA - B =")
print(A - B)

print("\nA * B =")
print(A * B)

print("\nA / B =")
print(A / B)

print("\nExplanation:")
print("These operations are element-wise because A and B have the same shape.")
print("Each entry in A combines with the matching entry in B.")

def problem4_manual_tensor_addition():
    section("Problem 4 - Manual Tensor Addition")

    A = np.array([
        [[1, 2], [3, 4]],
        [[5, 6], [7, 8]]
    ])

    B = np.array([
        [[10, 20], [30, 40]],
        [[50, 60], [70, 80]]
    ])

    C_manual = np.zeros(A.shape, dtype=int)

    for level in range(A.shape[0]):
        for row in range(A.shape[1]):
            for col in range(A.shape[2]):
                C_manual[level, row, col] = A[level, row, col] + B[level, row, col]

    C_numpy = A + B

    print("Manual tensor addition C_manual =")
    print(C_manual)

    print("\NumPy tensor addition C_numpy =")
    print(C_numpy)

    print("\nDo they match?", np.array_equal(C_manual, C_numpy))

    print("\nExplanation:")
    print("The nested loops show exactly what NumPy does for element-wise addition.")
    print("The level loop selects a matrix slice, the row loop selects a row, and")
    print("the column loop selects a scalar entry.")

def problem5_tensor_product():
    section("Problem 5 - Tensor Product")

    a = np.array([1, 2, 3])
    b = np.array([4, 5])

    C = np.tensordot(a, b, axes=0)

    print("a =", a)
    print("b =", b)
    print("\nnp.tensordot(a, b, axes=0) =")
    print(C)
    print("C.shape =", C.shape)

    print("\nExplanation:")
    print("The tensor product of a length-3 vector and a length-2 vector creates")

```

```

print("a 3 by 2 matrix. Each entry is a[i] * b[j].")

def part_b_image_cube_analysis():
    section("Part B - Practical Problem: Image Cube Analysis")

    Video = np.array([
        [[10, 11, 12], [13, 14, 15], [16, 17, 18]],
        [[20, 21, 22], [23, 24, 25], [26, 27, 28]],
        [[30, 31, 32], [33, 34, 35], [36, 37, 38]],
        [[40, 41, 42], [43, 44, 45], [46, 47, 48]]
    ], dtype=float)

    print("Video.shape =", Video.shape)
    print("Interpretation: 4 frames, 3 rows per frame, 3 columns per row.")

    # Average brightness of each frame: average over rows and columns.
    frame_means = Video.mean(axis=(1, 2))
    print("\nAverage brightness of each frame:")
    print(frame_means)

    overall_mean = Video.mean()
    print("\nAverage brightness of entire video:")
    print(overall_mean)

    brightest_pixel = Video.max()
    brightest_index = np.unravel_index(np.argmax(Video), Video.shape)
    print("\nBrightest pixel value:")
    print(brightest_pixel)
    print("Brightest pixel index [frame,row,column]:")

    print(brightest_index)
    print("Frame containing brightest pixel:", brightest_index[0])

    Video_brightened = Video * 1.25
    print("\nBrightened video tensor =")
    print(Video_brightened)

    print("\nExplanation:")
    print("A single grayscale image is naturally a matrix: rows by columns.")
    print("A grayscale video is a stack of images over time, so it needs three")
    print("dimensions: frame, row, and column. That is why a tensor is the right")
    print("data structure.")

def challenge_question():
    section("Challenge Question - Deep Learning Image Tensor Dimensions")

    print("In deep learning, image batches are often stored as:")
    print("(batch, height, width, channels)")
    print("\nMeaning:")
    print("batch    = number of images processed together")
    print("height   = number of pixel rows")
    print("width    = number of pixel columns")
    print("channels = color channels, usually 3 for RGB")
    print("\nA matrix is insufficient for color images because a color image needs")
    print("height, width, and channel dimensions. A batch of color images needs")
    print("one more dimension for the batch.")

def main():
    problem1_create_tensor()
    problem2_tensor_indexing()
    problem3_tensor_arithmetic()
    problem4_manual_tensor_addition()
    problem5_tensor_product()
    part_b_image_cube_analysis()
    challenge_question()

if __name__ == "__main__":
    main()

```

Expected Console Output

Problem 1 - Creating Tensors

```
T =
[[[ 1  2  3]
  [ 4  5  6]]

 [[ 7  8  9]
  [10 11 12]]]
T.shape = (2, 2, 3)
Number of levels = 2
Number of rows per level = 2
Number of columns per row = 3
```

Explanation:

T is a 3D tensor. The first index selects the level, the second selects the row inside that level, and the third selects the column inside that row.

Problem 2 - Tensor Indexing

```
First level T[0] =
[[1 2 3]
 [4 5 6]]
```

```
Second level T[1] =
[[ 7  8  9]
 [10 11 12]]
```

```
T[1, 0, 2] = 9
```

Explanation:

T[1, 0, 2] means: level 1, row 0, column 2.
Python uses zero-based indexing, so level 1 is the second level, row 0 is the first row, and column 2 is the third column.

Problem 3 - Tensor Arithmetic

```
A.shape = (2, 2, 2)
B.shape = (2, 2, 2)
```

```
A + B =
[[[11. 22.]
  [33. 44.]]

 [[55. 66.]
  [77. 88.]]]
```

```
A - B =
[[[ -9. -18.]
  [-27. -36.]]

 [[-45. -54.]
  [-63. -72.]]]
```

```
A * B =
[[[ 10.  40.]
  [ 90. 160.]]

 [[250. 360.]
  [490. 640.]]]
```

```
A / B =
[[[0.1 0.1]
  [0.1 0.1]]

 [[0.1 0.1]
  [0.1 0.1]]]
```

Explanation:

These operations are element-wise because A and B have the same shape. Each entry in A combines with the matching entry in B.

Problem 4 - Manual Tensor Addition

```
Manual tensor addition C_manual =
[[[11 22]
```

```
[33 44]]

[[55 66]
 [77 88]]]
```

```
NumPy tensor addition C_numpy =
[[[11 22]
  [33 44]]
```

```
[[55 66]
 [77 88]]]
```

Do they match? True

Explanation:
The nested loops show exactly what NumPy does for element-wise addition.
The level loop selects a matrix slice, the row loop selects a row, and
the column loop selects a scalar entry.

```
=====
Problem 5 - Tensor Product
=====
```

```
a = [1 2 3]
b = [4 5]
```

```
np.tensordot(a, b, axes=0) =
[[ 4  5]
 [ 8 10]
 [12 15]]
C.shape = (3, 2)
```

Explanation:
The tensor product of a length-3 vector and a length-2 vector creates
a 3 by 2 matrix. Each entry is $a[i] * b[j]$.

```
=====
Part B - Practical Problem: Image Cube Analysis
=====
```

```
Video.shape = (4, 3, 3)
Interpretation: 4 frames, 3 rows per frame, 3 columns per row.
```

```
Average brightness of each frame:
[14. 24. 34. 44.]
```

```
Average brightness of entire video:
29.0
```

```
Brightest pixel value:
48.0
Brightest pixel index [frame,row,column]:
(np.int64(3), np.int64(2), np.int64(2))
Frame containing brightest pixel: 3
```

```
Brightened video tensor =
[[[12.5 13.75 15.  ]
  [16.25 17.5 18.75]
  [20.   21.25 22.5 ]]

 [[25.   26.25 27.5 ]
  [28.75 30.   31.25]
  [32.5  33.75 35.   ]]

 [[37.5  38.75 40.   ]
  [41.25 42.5  43.75]
  [45.   46.25 47.5  ]]

 [[50.   51.25 52.5 ]
  [53.75 55.   56.25]
  [57.5  58.75 60.   ]]]
```

Explanation:
A single grayscale image is naturally a matrix: rows by columns.
A grayscale video is a stack of images over time, so it needs three
dimensions: frame, row, and column. That is why a tensor is the right
data structure.

```
=====
Challenge Question - Deep Learning Image Tensor Dimensions
=====
```

```
In deep learning, image batches are often stored as:
(batch, height, width, channels)
```

Meaning:

batch = number of images processed together
height = number of pixel rows
width = number of pixel columns
channels = color channels, usually 3 for RGB

A matrix is insufficient for color images because a color image needs height, width, and channel dimensions. A batch of color images needs one more dimension for the batch.